Artificial Intelligence: Tutorial-7

Name: Gaurav Damor

Admission Number: U24AI026

GITHUB LINK - [AI-LAB/LAB-7 at main · GAURAVSVNIT/AI-LAB]

The 8 queen's problem is stated as: Place 8 queens on a chessboard such that no two queens attack each other. Considering this problem, complete the following tasks considering a valid heuristic function with justification.

Q1 Generate 50 random initial boards. Run steepest -ascent hill climbing on each and record initial heuristic value, final heuristic value, number of steps taken and status solved or fail. Prove the presence of a local minimum.

CODE:

```
import random
import matplotlib.pyplot as plt

def calc(arr):
    h = 0
    for i in range(8):
        for j in range(i + 1, 8):
            if arr[i] == arr[j]:
                h = h + 1
            if abs(arr[i] - arr[j]) == abs(i - j):
                h = h + 1
    return h

def get_nxt(b):
    min_h = calc(b)
    best = list(b)

    for i in range(8):
        for j in range(8):
            if b[i] != j:
                temp = list(b)
                temp[i] = j
                th = calc(temp)
                if th < min_h:
                    min_h = th
                    best = list(temp)
    return best, min_h

def run_hc(b):
```

```
    cur = b
    h1 = calc(cur)
    st = 0

    while 1:
        n_b, n_h = get_nxt(cur)
        if n_h >= h1:
            break
        cur = n_b
        h1 = n_h
        st = st + 1

    return cur, h1, st

def show_b(arr):
    fig, ax = plt.subplots()
    for i in range(9):
        ax.axhline(i - 0.5, color='black', lw=1)
        ax.axvline(i - 0.5, color='black', lw=1)

    for c in range(8):
        r = arr[c]
        ax.text(c, r, 'Q', fontsize=20, ha='center', va='center', color='red')

    ax.set_xlim(-0.5, 7.5)
    ax.set_ylim(-0.5, 7.5)
    ax.invert_yaxis()
    plt.title("Solved 8 Queens")
    plt.show()

print("Run\tInit H\tFinal H\tSteps\tStatus")

fails = 0
solved = 0

for x in range(50):
    start_b = []
    for k in range(8):
        start_b.append(random.randint(0, 7))

    start_val = calc(start_b)
    e, v, s = run_hc(start_b)

    stat = "Fail"
    if v == 0:
        stat = "Solved"
        solved = solved + 1
    else:
        fails = fails + 1

    print(str(x+1) + "\t" + str(start_val) + "\t" + str(v) + "\t" + str(s) + "\t" + stat)

print("Failed cases prove local minimum.")
```

```python
# solve it
print("\nSolving with restarts...")
r_c = 0
while 1:
    b2 = []
    for k in range(8):
        b2.append(random.randint(0, 7))
    e2, v2, s2 = run_hc(b2)
    if v2 == 0:
        print("Done")
        print(e2)
        show_b(e2)
        break
    r_c = r_c + 1
```

OUTPUT:

Run	Init H	Final H	Steps	Status
1	4	1	3	Fail
2	5	3	1	Fail
3	5	0	3	Solved
4	7	2	2	Fail
5	10	0	4	Solved
6	6	2	2	Fail
7	4	2	1	Fail
8	7	1	4	Fail
9	6	0	3	Solved
10	10	0	5	Solved
11	10	0	5	Solved
12	5	2	2	Fail
13	8	3	2	Fail
14	5	1	3	Fail
15	7	2	3	Fail
16	9	1	4	Fail

17	7	0	4	Solved
18	3	1	1	Fail
19	7	1	5	Fail
20	10	2	3	Fail
21	9	1	3	Fail
22	6	2	2	Fail
23	4	0	3	Solved
24	5	1	3	Fail
25	5	1	2	Fail
26	7	1	4	Fail
27	6	2	3	Fail
28	6	1	3	Fail
29	9	0	4	Solved
30	10	1	4	Fail
31	9	1	4	Fail
32	11	1	4	Fail
33	3	2	1	Fail
34	9	0	4	Solved
35	8	0	5	Solved
36	11	2	3	Fail
37	9	1	4	Fail
38	12	1	4	Fail
39	6	1	2	Fail
40	6	0	4	Solved
41	6	1	3	Fail
42	11	3	3	Fail

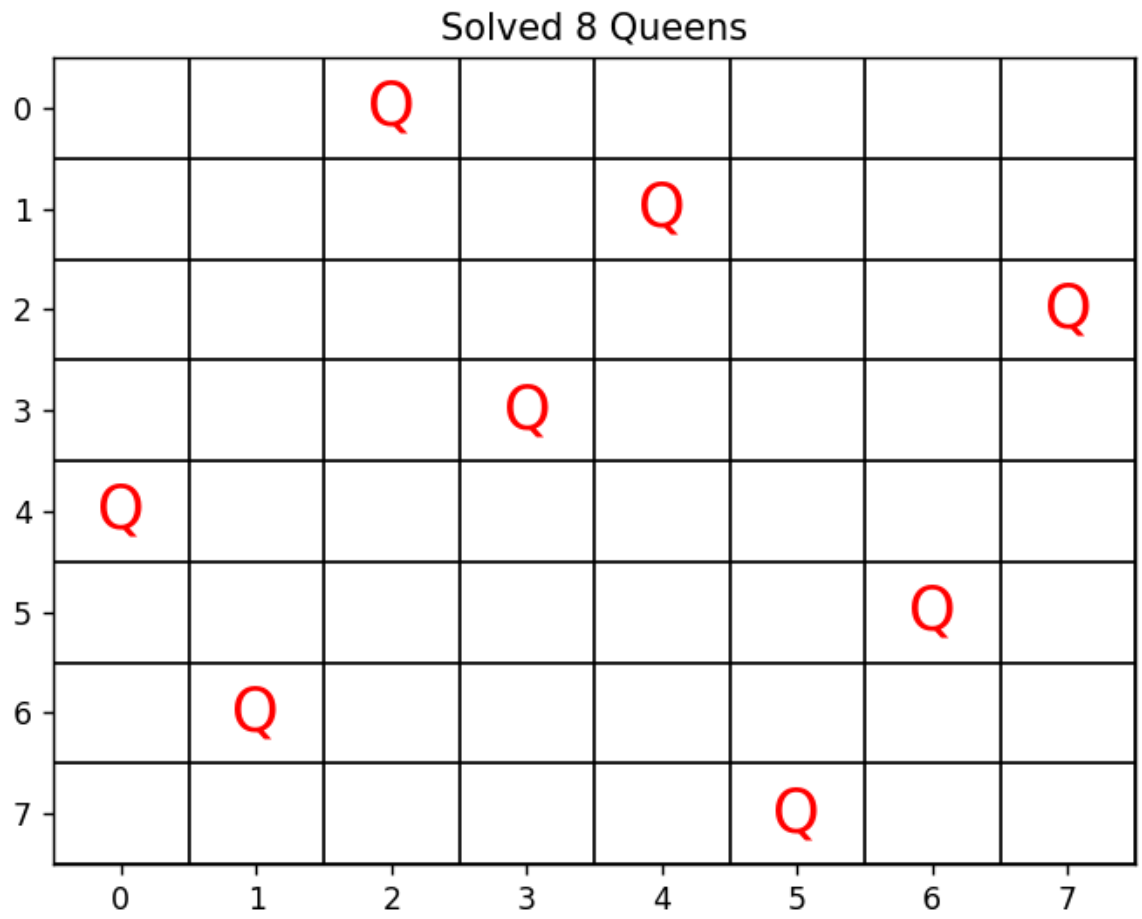
43	**9**	**1**	**4**	**Fail**
44	**6**	**2**	**3**	**Fail**
45	**6**	**1**	**2**	**Fail**
46	**8**	**1**	**3**	**Fail**
47	**2**	**1**	**1**	**Fail**
48	**6**	**1**	**3**	**Fail**
49	**4**	**2**	**1**	**Fail**
50	**17**	**1**	**5**	**Fail**

Failed cases prove local minimum.

Solving with restarts...

Done

[2, 4, 1, 7, 0, 6, 3, 5]



Q2 Repeat the above experiment on first choice, random restart and simulated annealing variants of hill climbing and compare your results.

CODE:

```
# Q2 Repeat the above experiment on first choice, random restart and simulated annealing
# variants of hill climbing and compare your results.

import random
import math

def calc(arr):
    h = 0
    for i in range(8):
        for j in range(i + 1, 8):
            if arr[i] == arr[j]:
                h = h + 1
            if abs(arr[i] - arr[j]) == abs(i - j):
                h = h + 1
    return h

def get_rnd_b():
    b = []
    for k in range(8):
```

```
        b.append(random.randint(0, 7))
    return b

def run_fc(b):
    # first choice hc
    cur = b
    h1 = calc(cur)
    st = 0

    while 1:
        # get random neighbor
        temp = list(cur)
        c = random.randint(0, 7)
        r = random.randint(0, 7)
        temp[c] = r

        n_h = calc(temp)
        if n_h < h1:
            cur = temp
            h1 = n_h
            st = st + 1
        else:
            # stop if stuck for a while (hacky way to check local minimum)
            if st > 1000:
                break
            st = st + 1

    return cur, h1, st

def run_rr():
    # random restart
    st_tot = 0
    rst = 0

    while 1:
        b = get_rnd_b()
        cur = b
        h1 = calc(cur)

        # steepest ascent internal
        while 1:
            best_n = list(cur)
            min_h = h1
            for i in range(8):
                for j in range(8):
                    if cur[i] != j:
                        temp = list(cur)
                        temp[i] = j
                        th = calc(temp)
                        if th < min_h:
                            min_h = th
                            best_n = list(temp)

            if min_h >= h1:
```

```
                break

            cur = best_n
            h1 = min_h
            st_tot = st_tot + 1

        rst = rst + 1
        if h1 == 0:
            return cur, h1, st_tot, rst

def run_sa(b):
    # simulated annealing
    cur = b
    h1 = calc(cur)
    st = 0
    t = 100.0
    dr = 0.99

    while t > 0.001:
        if h1 == 0:
            break

        temp = list(cur)
        c = random.randint(0, 7)
        r = random.randint(0, 7)
        temp[c] = r
        n_h = calc(temp)

        de = h1 - n_h

        if de > 0:
            cur = list(temp)
            h1 = n_h
        else:
            p = math.exp(de / t)
            if random.random() < p:
                cur = list(temp)
                h1 = n_h

        t = t * dr
        st = st + 1

    return cur, h1, st

print("First Choice HC 50 Runs")
fc_s = 0
fc_f = 0
fc_st = 0
for x in range(50):
    b = get_rnd_b()
    e, v, s = run_fc(b)
    fc_st = fc_st + s
    if v == 0:
        fc_s = fc_s + 1
```

```
    else:
        fc_f = fc_f + 1
print("Solved: " + str(fc_s) + " Fails: " + str(fc_f) + " Avg steps: " + str(fc_st/50.0))

print("\nRandom Restart (runs until solved)")
e2, v2, s2, r_cnt = run_rr()
print("Solved in " + str(r_cnt) + " restarts with " + str(s2) + " total steps")

print("\nSimulated Annealing 50 Runs")
sa_s = 0
sa_f = 0
sa_st = 0
for y in range(50):
    b = get_rnd_b()
    e3, v3, s3 = run_sa(b)
    sa_st = sa_st + s3
    if v3 == 0:
        sa_s = sa_s + 1
    else:
        sa_f = sa_f + 1
print("Solved: " + str(sa_s) + " Fails: " + str(sa_f) + " Avg steps: " + str(sa_st/50.0))
```

OUTPUT:

First Choice HC 50 Runs

Solved: 5 Fails: 45 Avg steps: 1001.0

Random Restart (runs until solved)

Solved in 3 restarts with 10 total steps

Simulated Annealing 50 Runs

Solved: 29 Fails: 21 Avg steps: 946.66